



ConsultIQ

Testing Policy Document

updated: July 30, 2026

Contents

1	Introduction	2
2	Testing Objectives	2
3	Scope	2
4	Responsibilities	3
5	Testing Types	3
5.1	Unit Testing	3
5.2	Integration Testing	3
5.3	UI Testing (Playwright)	4
5.4	End-to-End (E2E) Testing	5
5.5	Manual Testing	6
5.6	Regression Testing	7
6	Testing Tools and Environments	8
6.1	Test Environments	8
7	Test Structure and Conventions	9
7.1	Naming Conventions	9
7.2	Directory Structure	9
7.3	Coverage Targets	9
7.4	Test Commands	10
8	Defect Management	10
8.1	Defect Discovery	10
8.2	Defect Classification	11
8.3	Defect Workflow	11
9	Acceptance Criteria	11
10	CI/CD Integration	11
11	Reporting and Review	13
11.1	Coverage Reports	13
11.2	Test Review Cadence	13
11.3	Document Maintenance	13

1 Introduction

The Testing Policy Document covers every layer of the system — the NestJS backend, the React/TypeScript frontend, the PostgreSQL database, and the CI/CD pipeline — and applies to all testing activity carried out by Team TROOS.

Our testing strategy includes the following:

- Unit Testing (Backend and Frontend) with Jest
- Integration Testing with Jest against a real PostgreSQL instance
- UI Testing with Playwright, against a real running frontend with all network calls mocked
- End-to-End (E2E) Testing with Playwright, exercising the full stack — real backend, real database, real UI
- Manual Testing for UI behaviour and API verification

The following sections describe the tools, procedures, and standards applied to achieve reliable, high-quality results.

2 Testing Objectives

Testing on ConsultIQ is aimed at a few concrete goals rather than testing for its own sake:

- Verify that implemented features conform to the functional requirements defined in the Software Requirements Specification (SRS).
- Detect and document defects early in the development cycle, while they are still cheap to fix.
- Ensure that new code changes do not break existing functionality (regression prevention).
- Validate that the system behaves correctly under realistic conditions, including both valid and invalid input.
- Enforce a minimum standard of test coverage across backend services and controllers.
- Confirm that the frontend and backend are correctly wired together end to end, not just correct in isolation.

3 Scope

The table below summarises which parts of the system fall under this policy.

Layer	Components
Backend API	NestJS services, controllers, repositories, DTOs, guards, and interceptors
Frontend	React components, service functions, and the API integration layer
Database	Prisma ORM query correctness, schema integrity, and integration testing against a real PostgreSQL instance
CI/CD Pipeline	Automated test execution on every pull request via GitHub Actions
Authentication	JWT-based auth flows, role-based access control, token refresh and revocation

Layer	Components
Scoring Engine	Match-run execution, weight resolution, factor scoring modules, and result aggregation
UI / E2E (Playwright)	Frontend rendering and behaviour against mocked data (UI tests), and full-stack login, scoring, and dashboard rendering against a real backend and database (E2E tests)

4 Responsibilities

Testing is a shared responsibility rather than something delegated to a single team member. No code is merged into `dev` without passing all automated checks and receiving at least one peer review approval, and any team member is expected to flag gaps in coverage or missing tests during review — not just the author of the branch.

5 Testing Types

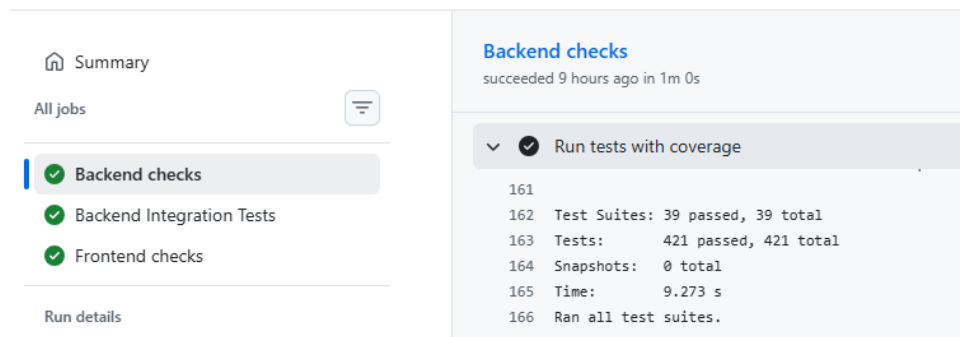
5.1 Unit Testing

Unit tests validate individual components in isolation, ensuring that each small, independent part of the system functions correctly.

Applies to: Backend NestJS services, controllers, guards, validators, scoring modules

Framework: Jest with `ts-jest`

Location: Files matching `*.spec.ts` in `backend/src/`



Conventions:

- Every service and controller must have a corresponding `*.spec.ts` file.
- Tests must mock all Prisma calls using `jest.fn()` — no real database connections.
- Each `describe` block maps to one method or logical grouping of behaviour.
- Every `it` block tests exactly one behaviour: a successful path, an error case, or an edge case.

5.2 Integration Testing

Integration tests check that multiple components work correctly together, including real interaction with the database. These run against a dedicated PostgreSQL database, `consultiq_test`, which is wiped clean before every run.

Applies to: Backend service methods that interact with the database
Framework: Jest with ts-jest, dotenv-cli, real PrismaService
Config: jest-int.json, -runInBand flag (prevents deadlocks from parallel TRUNCATE)
Location: Files matching *.int-spec.ts in backend/src/
Environment: .env.test pointing to the consultiq_test database

```
✓ Run Integration Tests

1 ▶ Run npm run test:int
4
5 > backend@0.0.1 test:int
6 > dotenv -e .env.test -- jest --config jest-int.json --runInBand
7
8 PASS src/scoring/services/match-run.service.int-spec.ts
9 PASS src/admin/users/services/admin.user.service.int-spec.ts
10 PASS src/scoring/services/scoring-config.service.int-spec.ts
11
12 Test Suites: 3 passed, 3 total
13 Tests:      17 passed, 17 total
14 Snapshots:  0 total
15 Time:       3.005 s
16 Ran all test suites.
```

Conventions:

- beforeAll bootstraps the full NestJS module — no mocks.
- beforeEach calls cleanDatabase(prisma) to wipe all tables via TRUNCATE CASCADE.
- Each test explicitly seeds its own data.
- No state is shared between tests; each one is fully self-contained.
- Run sequentially with npm run test:int — not in parallel.

Setup commands:

```
# First-time setup
npm run test:int:setup

# Subsequent runs
npm run test:int

# Single file
npm run test:int -- admin.user.service.int-spec.ts
```

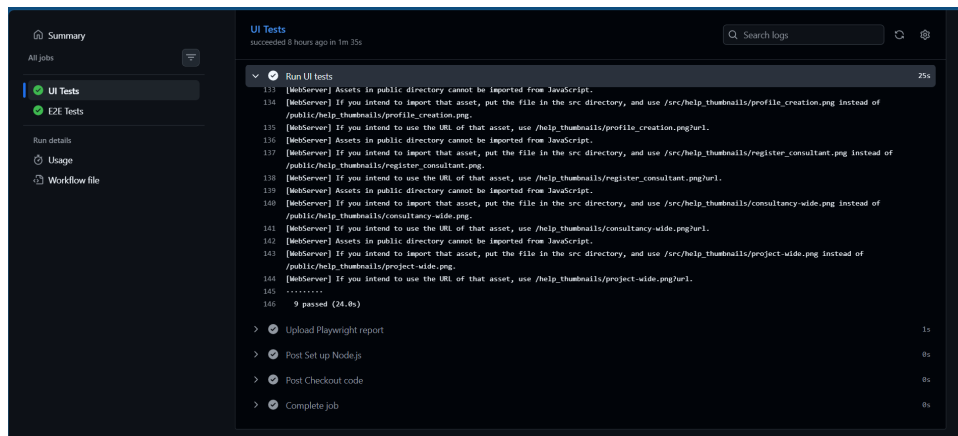
5.3 UI Testing (Playwright)

UI tests validate the frontend's own rendering and behaviour — do components render correctly given data, and do they degrade gracefully when data is missing or an API call fails — without depending on a real backend or database. Because no test mutates shared state, the UI suite runs safely in full parallel across multiple browser engines.

Applies to: React components, pages, and client-side rendering/error-handling logic
Framework: Playwright Test (@playwright/test)
Config: playwright.ui.config.ts
Location: Files matching *.spec.ts in tests/ui/
Backend/DB required: No — all network calls are mocked via page.route()

Conventions:

- Runs across chromium, firefox, and webkit, with fullyParallel: true.
- Only the frontend (Vite dev server) is started via webServer, since every backend call is mocked.
- expect.timeout is raised to 10000 ms (from Playwright's 5s default) to absorb first-load/cold-start compile time.
- Endpoints such as /auth/me and match-run stats are mocked directly in each spec via page.route().
- Specs assert that stat values (Total Evaluated, Placements, Excluded) render correctly from mocked data, and that the dashboard fails gracefully — no crash, no NaN/undefined rendered — when the stats API call errors.
- Recommendations passed via React Router's location.state are exercised by injecting state through window.history.pushState plus a synthetic popstate event, since page.goto() cannot carry in-memory router state.
- A spec also asserts the recommendations table renders empty when no location state is provided.



5.4 End-to-End (E2E) Testing

E2E tests exercise the full application stack — real backend, real database, real frontend — the way an actual user would interact with the system. Because Jest's integration suite already covers the scoring engine's correctness in detail, E2E tests are deliberately narrow: their job is to confirm the full stack is wired together correctly, not to re-prove business logic.

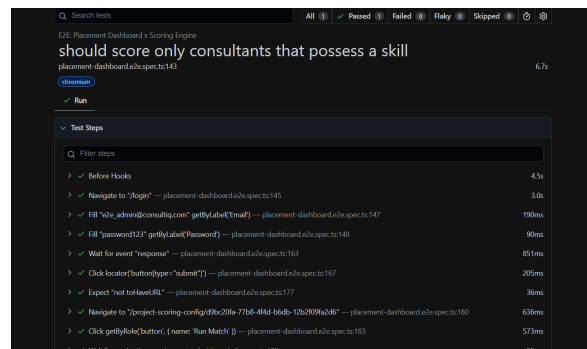
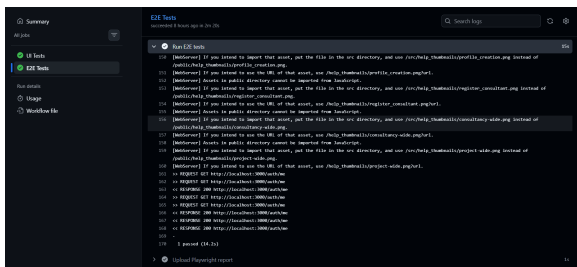
Applies to: The full stack — NestJS backend, PostgreSQL test database, and the React frontend, wired together
Framework: Playwright Test (@playwright/test)
Config: playwright.e2e.config.ts

Location: Files matching `*.e2e.spec.ts` in `tests/e2e/`

Backend/DB required: Yes — a real running NestJS server and a real PostgreSQL test database, seeded via Prisma before each run

Conventions:

- Runs `chromium` only, with `workers: 1` and `fullyParallel: false` — required because parallel workers seeding the same fixture emails into the same database causes unique-constraint failures.
- Both the backend (NestJS) and frontend (Vite) are started via a `webServer` array, and Playwright waits for each to respond before running any test.
- `backend/.env.test` is loaded via `dotenv` at config-load time for local runs; in CI, job-level `env:` values (from repo secrets) take precedence automatically, since `dotenv.config()` never overwrites a variable already present in `process.env`.
- All required connection values (`DATABASE_URL`, `REDIS_URL`, `JWT_SECRET`, `RESEND_API_KEY`) are read via a `requireEnv()` helper that throws immediately if any are missing — no hard-coded fallback credentials exist in this file.
- `beforeAll` seeds a real test database: an admin user, a project with a required skill (`ProjectSkillRequirement`), and three consultants with deliberately different skill profiles (`Perfect Match`, `Partial Match`, `No Match` — the last with no matching skill at all, to exercise the engine's exclusion logic).
- Login is performed through the real UI login form (not a mocked auth call), asserting the real backend's response.
- The test navigates to the real scoring-trigger page (`/project-scoring-config/:projectId`) and clicks the real **Run Match** button.
- Assertions confirm the real scoring engine correctly evaluates all three consultants, correctly excludes the unskilled one from the recommendations table, and that the stats grid reflects the right evaluated/excluded counts.
- `afterAll` cleans up all seeded data via `cleanDatabase()`.



5.5 Manual Testing

Manual testing is performed by team members to validate UI behaviour, user flows, and scenarios that are difficult to automate — and to catch anything the automated UI and E2E suites do not yet cover.

Applies to: Frontend pages, navigation flows, form validation, visual correctness
Tools: Browser DevTools
When performed: Before opening a pull request for frontend-heavy changes

5.6 Regression Testing

Regression testing ensures that new changes do not break previously working functionality. This is enforced automatically through the CI/CD pipeline, which runs the full unit test suite, the UI (Playwright) suite, and the E2E (Playwright) suite on every pull request. Any PR that causes previously passing tests to fail must be fixed before merging.

6 Testing Tools and Environments

Tool	Purpose
Jest	Unit and integration test runner
ts-jest	TypeScript transformer for Jest
dotenv-cli / dotenv	Environment variable injection for integration and E2E tests
Playwright (@playwright/test)	UI and E2E browser test runner, used for both the mocked-network UI suite and the full-stack E2E suite
bcrypt	Hashes seeded admin/consultant passwords at the same cost factor (12) used by <code>auth.credential.service.ts</code> , so login against seeded E2E fixtures succeeds
GitHub Actions	CI/CD pipeline — automated test execution on pull requests
SonarCloud	Static code analysis, cognitive complexity enforcement, and secret-detection (flags any hardcoded connection string)
Docker	Local PostgreSQL (<code>consultiq_db</code>) and Redis (<code>consultiq-redis</code>) containers; also used to provision Postgres/Redis as CI-only service containers for E2E tests

6.1 Test Environments

Environment	Purpose	Database
Local	Active development and manual testing	<code>consultiq</code> (local Docker container, port 5432)
Development	Isolated integration test runs	<code>consultiq_test</code> (separate database, wiped per run)
Integration Test	Frontend rendering/behaviour against mocked API responses	None — only a Vite dev server is started
UI Test (Playwright)	Full-stack login, scoring, and rendering, seeded via Prisma	<code>consultiq_test</code> , backed locally by a native (non-containerized) Postgres/Redis install, or by Docker service containers in CI
E2E Test (Playwright)	Automated checks on every PR	Unit tests run mocked (no DB); E2E job provisions <code>postgres:15</code> and <code>redis:7</code> as service containers
CI (GitHub Actions)	Live deployed application	Managed PostgreSQL on AWS
Production (EC2)		

7 Test Structure and Conventions

7.1 Naming Conventions

File type	Pattern
Unit test	*.spec.ts (in backend/src/)
Integration test	*.int-spec.ts
UI test (Playwright)	*.spec.ts (in tests/ui/)
E2E test (Playwright)	*.e2e.spec.ts (in tests/e2e/)

7.2 Directory Structure

Tests are separated by folder specifically so that CI — and any developer running them — never has to guess which suite needs a real backend/database and which does not.

```
tests/
|-- ui/
|   '-- placement-dashboard.spec.ts      <- mocked-network UI test
'-- e2e/
    '-- placement-dashboard.e2e.spec.ts  <- real-backend E2E test

backend/
|-- src/
|   |-- projects/
|   |   '-- services/
|   |       |-- project.service.ts
|   |       '-- project.service.spec.ts      <- unit test
|   |-- admin/
|   |   '-- users/
|   |       '-- services/
|   |           |-- admin.user.service.ts
|   |           |-- admin.user.service.spec.ts  <- unit test
|   |           '-- admin.user.service.int-spec.ts <- integration test
|   '-- scoring/
|       '-- services/
|           |-- match-run.service.ts
|           '-- match-run.service.int-spec.ts  <- integration test
|-- test/
|   '-- app.e2e-spec.ts                    <- legacy Jest E2E test
|-- prisma/
|   '-- prisma-test-utils.ts                <- cleanDatabase() and test DB helpers
'-- jest-int.json                           <- integration test config
```

7.3 Coverage Targets

Layer	Min. statement coverage
Backend services	80%
Backend controllers	80%
Scoring engine modules	80%
Frontend service functions	Not enforced

Layer	Min. statement coverage
UI / E2E (Playwright)	Not measured by statement coverage — tracked by scenario checklist instead

Coverage is measured using `npm run test:cov`.

7.4 Test Commands

All commands are exposed as npm scripts (root `package.json` unless noted), rather than invoked via raw `npx` in CI — this keeps every invocation pinned to the exact version already installed via `npm ci / package-lock.json`, with no on-demand package fetching.

Command	What it does
<code>npm run test:ui</code>	Runs the UI suite (<code>playwright.ui.config.ts</code>)
<code>npm run test:e2e</code>	Runs the E2E suite (<code>playwright.e2e.config.ts</code>)
<code>npm run playwright:install</code>	Installs all three browser engines + OS deps
<code>npm run playwright:install:chromium</code>	Installs chromium only (used in the E2E CI job)
<code>npm run prisma:generate</code> (backend/)	Regenerates the Prisma client — required after <code>npm ci -ignore-scripts</code>
<code>npm run prisma:migrate:deploy</code> (backend/)	Applies pending migrations to the test database before E2E tests run

Interactive / debugging variants:

```
npx playwright test --config=playwright.ui.config.ts --ui
npx playwright test --config=playwright.e2e.config.ts --headed --project=chromium
npx playwright show-report
```

8 Defect Management

8.1 Defect Discovery

Defects may be discovered through:

- Failing automated tests (CI pipeline failures — unit, integration, UI, or E2E)
- Manual testing during development
- Peer code review on GitHub pull requests
- SonarCloud static analysis warnings

8.2 Defect Classification

Severity	Description
Critical	System cannot function; data loss or security breach
High	Core feature broken; no workaround available
Medium	Feature partially broken; workaround exists
Low	Minor cosmetic or non-functional issue

8.3 Defect Workflow

1. Defect is identified and documented as a GitHub Issue with an appropriate label (bug, plus severity level).
2. Issue is assigned to the responsible team member.
3. A fix branch is created (e.g. `hotfix/backend/fix-description`).
4. Fix is implemented with a corresponding test that reproduces the defect.
5. PR is opened, reviewed, and merged only after the test passes.
6. Issue is closed and linked to the merged PR.

9 Acceptance Criteria

A feature is considered complete and ready for merging when all of the following conditions are met:

- ☐ All unit tests pass (`npm run test` — 0 failures).
- ☐ Build succeeds with no TypeScript errors (`npm run build`).
- ☐ Linter passes with no errors (`npm run lint` — warnings from third-party code are acceptable).
- ☐ Type check passes (`npx tsc -noEmit`).
- ☐ All integration tests pass for affected services (`npm run test:int`).
- ☐ All UI tests pass for affected frontend components (`npm run test:ui`).
- ☐ All E2E tests pass for affected user flows (`npm run test:e2e`).
- ☐ At least one peer team member has reviewed and approved the pull request on GitHub.
- ☐ No SonarCloud cognitive complexity violations (maximum 15 per function), and no hard-coded credentials flagged by secret-detection.
- ☐ New features include corresponding unit tests covering happy path, error cases, and edge cases.

10 CI/CD Integration

Every pull request targeting `dev` or `main` triggers the following automated checks via GitHub Actions.

Backend Checks (verify-backend job)

1. Install dependencies — `npm ci`
2. Generate Prisma client — `npx prisma generate`
3. Run unit tests with coverage — `npm test - --coverage`
4. Upload coverage report — stored as a GitHub Actions artifact
5. Run linter — `npm run lint`
6. Type check — `npx tsc -noEmit`
7. Build — `npm run build`

Frontend Checks (verify-frontend job)

1. Install dependencies — `npm ci`
2. Run linter — `npm run lint`
3. Build — `npm run build`

Playwright Pipeline (.github/workflows/playwright.yml)

Triggers: pull requests into dev or main, and `workflow_dispatch` (manual trigger via the Actions tab or `gh workflow run`, available on any branch once the workflow file exists on the default branch).

ui-tests job:

1. Installs root + frontend dependencies (`npm ci -ignore-scripts`)
2. Installs Playwright browsers, runs the UI suite across all three engines
3. Uploads the HTML report as a build artifact regardless of pass/fail

e2e-tests job:

1. Spins up `postgres:15` and `redis:7` as service containers, health-checked before use
2. Installs root, backend, and frontend dependencies
3. Explicitly regenerates the Prisma client (required due to `-ignore-scripts`)
4. Writes `backend/.env.test` (and a copy to `.env`) from job-level secrets
5. Runs Prisma migrations against the service Postgres container
6. Installs chromium only, then runs the E2E suite (single worker, single browser)
7. Uploads the HTML report as a build artifact regardless of pass/fail

Policy

- No PR may be merged to dev or main unless all CI checks pass, including the `ui-tests` and `e2e-tests` jobs.
- If a CI check fails due to a teammate's code (not the PR author's code), the PR author must coordinate with the relevant teammate to fix the issue before merging.
- The `main` branch is reserved for production-ready code and triggers automatic deployment to AWS EC2.

11 Reporting and Review

11.1 Coverage Reports

Coverage reports are generated automatically on every CI run and uploaded as GitHub Actions artifacts. Team members can download and review coverage reports, as well as the Playwright HTML report for the UI and E2E suites, from the Actions tab on GitHub. To generate a coverage report locally:

```
npm run test:cov
```

11.2 Test Review Cadence

- **Per PR:** All tests must pass before requesting review.
- **Per Sprint:** Team lead reviews overall coverage trends and flags any services falling below the minimum threshold.
- **Per Milestone:** A full test run (unit + integration + UI + E2E + manual) is performed before each demo submission.

11.3 Document Maintenance

This testing policy document is a living document. It must be updated whenever:

- A new testing type or tool is introduced.
- Minimum coverage thresholds are revised.
- New team members join or roles change.
- The CI/CD pipeline configuration changes significantly.

All updates must be reviewed and approved by the Team Lead before being committed to the repository.